In [3]:
```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Q1: Load the penguins dataset from seaborn into a DataFrame named df and display the first 5 rows

In [4]:
```
df = sns.load_dataset("penguins")

df.head()
```

Out[4]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_m
0	Adelie	Torgersen	39.1	18.7	181.0	3
1	Adelie	Torgersen	39.5	17.4	186.0	3
2	Adelie	Torgersen	40.3	18.0	195.0	3
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	3

Q2: Print the following

In [5]:
```
df.shape
```

Out[5]:
```
(344, 7)
```

In [9]:
```
df.columns
```

Out[9]:
```
Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',
       'flipper_length_mm', 'body_mass_g', 'sex'],
      dtype='object')
```

In [8]:
```
print(df.dtypes)
```

```
species               object
island                object
bill_length_mm       float64
bill_depth_mm        float64
flipper_length_mm    float64
body_mass_g          float64
sex                   object
dtype: object
```

Q3: Check for missing values in each column:

In [10]:
```
print(df.isnull().sum())
```

```
species               0
island                0
bill_length_mm        2
bill_depth_mm         2
flipper_length_mm     2
body_mass_g           2
sex                  11
dtype: int64
```

Q4: Create a cleaned copy called df_clean by:

In [12]:
```
df_clean = df.copy()

df_clean.drop_duplicates(inplace=True)
```

In [18]:
```
numeric_cols = df_clean.select_dtypes(include='number').columns
categorical_cols = df_clean.select_dtypes(include=['object', 'category'])

for col in numeric_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

for col in categorical_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])
```

In [19]:
```
print(df_clean.isnull().sum())
```

```
species              0
island               0
bill_length_mm       0
bill_depth_mm        0
flipper_length_mm    0
body_mass_g          0
sex                  0
dtype: int64
```

Q5: Filter the dataset to include only penguins from the island "Biscoe" .

In [21]:
```
df_biscoe = df_clean[df_clean["island"] == "Biscoe"]

df_biscoe.head()
```

Out[21]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mas
20	Adelie	Biscoe	37.8	18.3	174.0	34
21	Adelie	Biscoe	37.7	18.7	180.0	36
22	Adelie	Biscoe	35.9	19.2	189.0	38
23	Adelie	Biscoe	38.2	18.1	185.0	39
24	Adelie	Biscoe	38.8	17.2	180.0	38

Q6: Find all penguins with:

In [22]:

```
mean_body_mass = df_clean["body_mass_g"].mean()

heavy_males = df_clean[(df_clean["body_mass_g"] > mean_body_mass) &(df_cl

heavy_males.head()
```

Out[22]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_
7	Adelie	Torgersen	39.2	19.6	195.0	
9	Adelie	Torgersen	42.0	20.2	190.0	
14	Adelie	Torgersen	34.6	21.1	198.0	
17	Adelie	Torgersen	42.5	20.7	197.0	
39	Adelie	Dream	39.8	19.1	184.0	

Q7: Sort df_clean by:

1. body_mass_g (descending)

2. then bill_length_mm (ascending)

In [23]:

```
df_sorted = df_clean.sort_values(
    by=["body_mass_g", "bill_length_mm"],
    ascending=[False, True]
)

df_sorted.head(10)
```

Out[23]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_ma
237	Gentoo	Biscoe	49.2	15.2	221.0	6
253	Gentoo	Biscoe	59.6	17.0	230.0	6
337	Gentoo	Biscoe	48.8	16.2	222.0	6
297	Gentoo	Biscoe	51.1	16.3	220.0	6
299	Gentoo	Biscoe	45.2	16.4	223.0	5
331	Gentoo	Biscoe	49.8	15.9	229.0	5
233	Gentoo	Biscoe	48.4	14.6	213.0	5
235	Gentoo	Biscoe	49.3	15.7	217.0	5
335	Gentoo	Biscoe	55.1	16.0	230.0	5
295	Gentoo	Biscoe	48.6	16.0	230.0	5

Q8: Compute the following aggregations:

1. Mean body_mass_g per species . Sort values in descending order.
2. Mean bill_length_mm per (species , sex)
3. Count of records per island

In [24]:

```
mean_body_mass_species = (
    df_clean
    .groupby("species")["body_mass_g"]
    .mean()
    .sort_values(ascending=False)
)

print(mean_body_mass_species)

mean_bill_species_sex = (
    df_clean
    .groupby(["species", "sex"])["bill_length_mm"]
    .mean()
)

print(mean_bill_species_sex)

mean_bill_species_sex = (
    df_clean
    .groupby(["species", "sex"])["bill_length_mm"]
    .mean()
    .reset_index()
)

count_per_island = df_clean["island"].value_counts()

print(count_per_island)
```

```
species
Gentoo       5067.741935
Chinstrap    3733.088235
Adelie       3702.960526
Name: body_mass_g, dtype: float64
species    sex
Adelie     Female    37.257534
           Male      40.280380
Chinstrap  Female    46.573529
           Male      51.094118
Gentoo     Female    45.563793
           Male      49.164394
Name: bill_length_mm, dtype: float64
island
Biscoe       168
Dream        124
Torgersen     52
Name: count, dtype: int64
```

Q9: Feature engineering:

Create a new column bill_ratio = bill_length_mm / bill_depth_mm .

1. Add it to df_clean
2. Show summary statistics for bill_ratio
3. Identify which species has the highest average bill_rati

In [25]:
```
df_clean["bill_ratio"] = df_clean["bill_length_mm"] / df_clean["bill_dept

df_clean.head()
df_clean["bill_ratio"].describe()
```

Out[25]:
```
count    344.000000
mean       2.605438
std        0.495930
min        1.639810
25%        2.161882
50%        2.571484
75%        3.097572
max        3.612676
Name: bill_ratio, dtype: float64
```

In [26]:
```
avg_ratio_species = (
    df_clean
    .groupby("species")["bill_ratio"]
    .mean()
    .sort_values(ascending=False)
)

print(avg_ratio_species)

print(avg_ratio_species.idxmax())
```

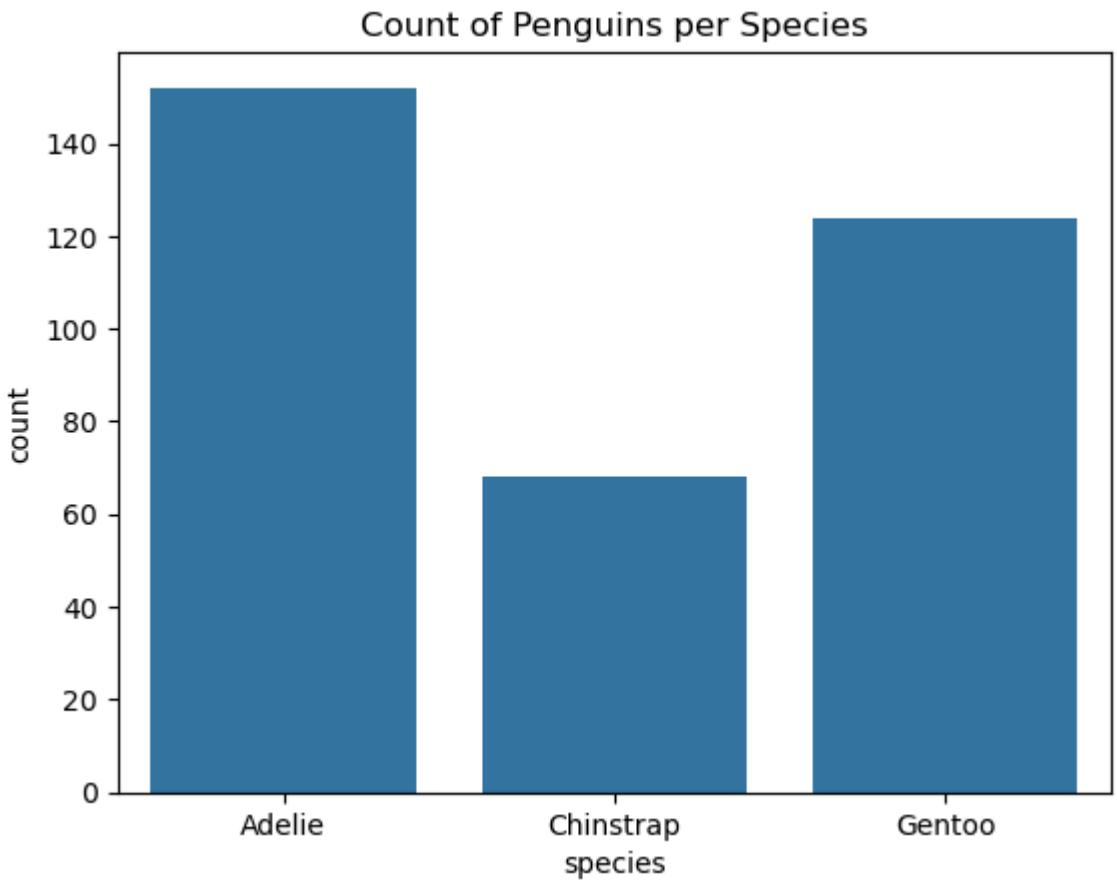
```
species
Gentoo       3.170703
Chinstrap    2.653756
Adelie       2.122684
Name: bill_ratio, dtype: float64
Gentoo
```

Q10: Create the following plots using df_clean :

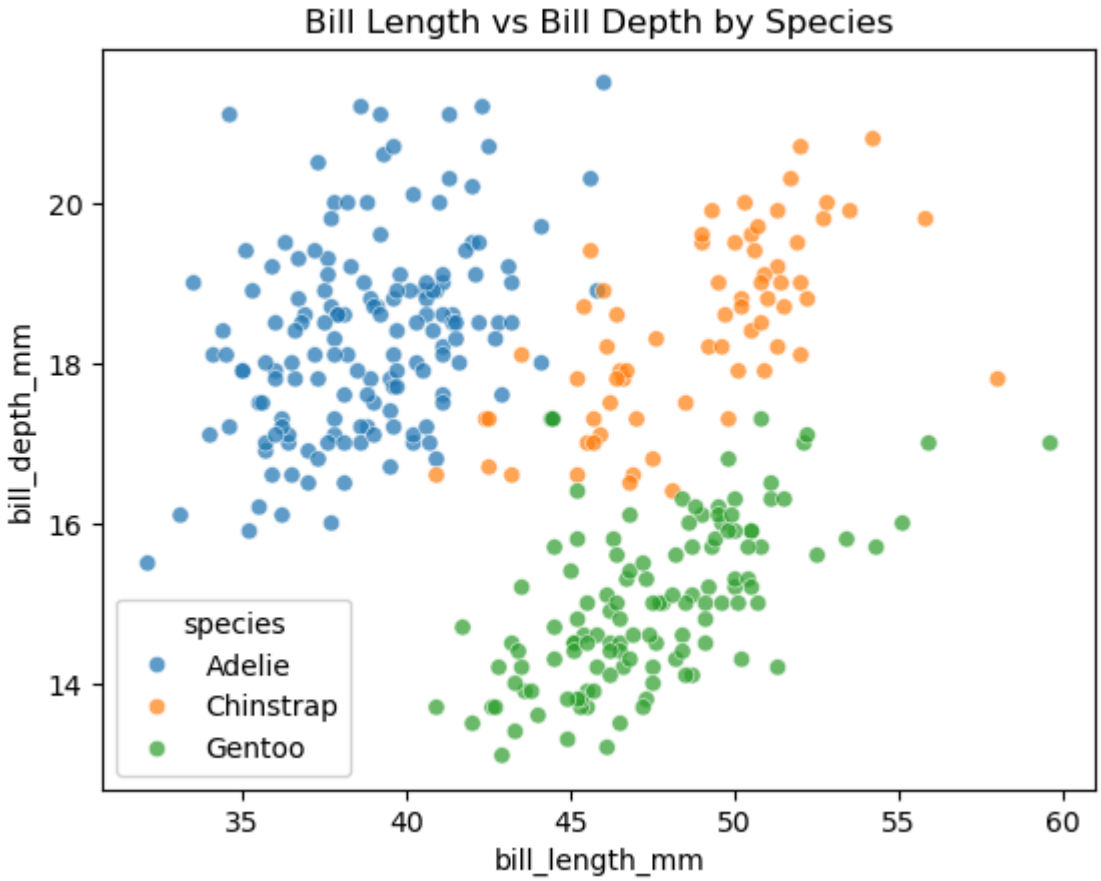
1. Bar plot: count of penguins per species
2. Scatter plot: bill_length_mm vs bill_depth_mm (color by species)

In [29]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure()
sns.countplot(data=df_clean, x="species")
plt.title("Count of Penguins per Species")
plt.show()
```



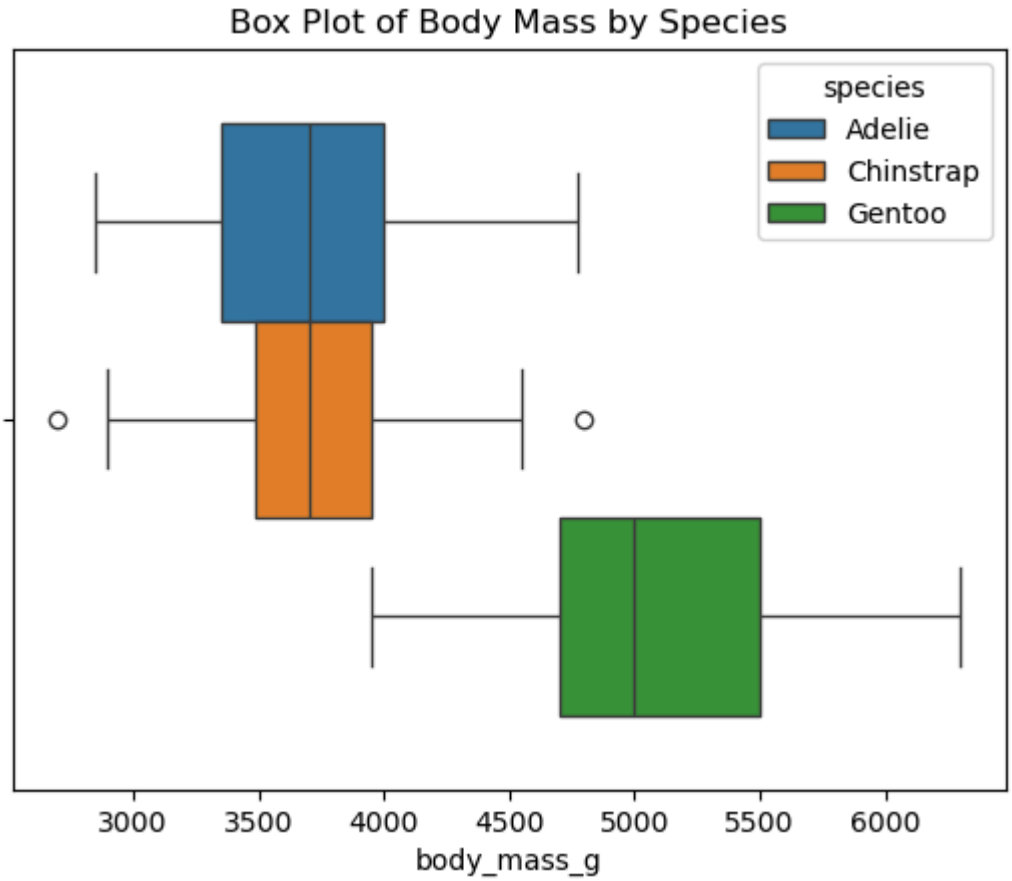
In [31]:
```
sns.scatterplot(x='bill_length_mm', y='bill_depth_mm', hue='species',
alpha=0.7, data=df_clean)
plt.title('Bill Length vs Bill Depth by Species')
plt.show()
```



Q11: Create a box plot of body_mass_g by species.

Then write 2–3 sentences interpreting what you observe

In [35]:
```
sns.boxplot(x='body_mass_g', hue='species', data=df_clean)
plt.title('Box Plot of Body Mass by Species')
plt.show()
```

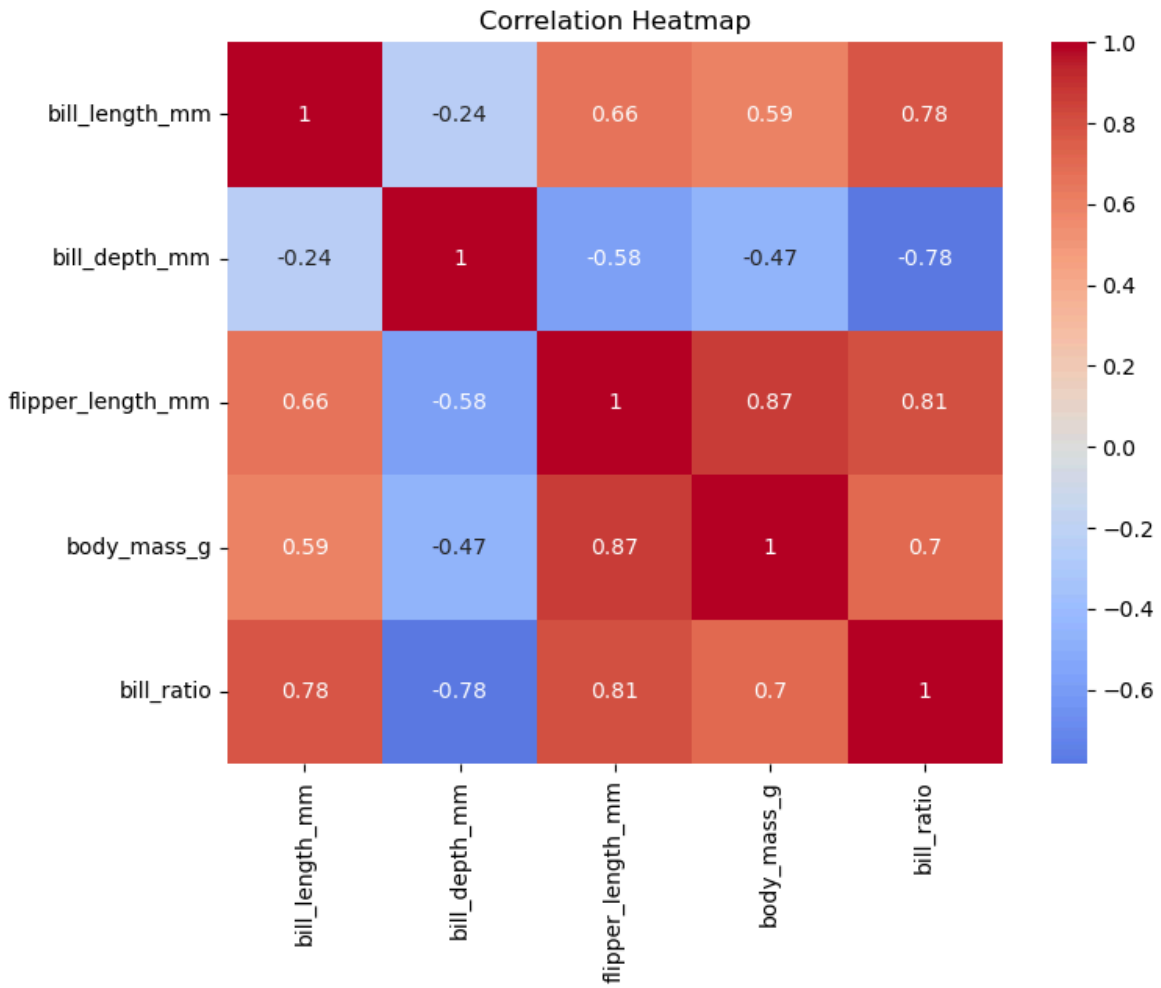


Body mass is being changed due to their species, Gentoo has the highest body mass, whereas Chinstrap has the lowest

Q12: Compute a correlation matrix for numeric features and visualize it as a heatmap.

In [37]:
```
import numpy as np

plt.figure(figsize=(8, 6))
numeric_cols = df_clean.select_dtypes(include=np.number).columns
sns.heatmap(df_clean[numeric_cols].corr(), annot=True, cmap='coolwarm',ce
plt.title('Correlation Heatmap')
plt.show()
```



Q13: Define:

target y = species features X = all other columns EXCEPT species Then split into train/test using: test_size = 0.2 random_state = 42 stratify by y

In [39]:
```
from sklearn.model_selection import train_test_split

# Define target and features
y = df_clean["species"]
X = df_clean.drop(columns=["species"])

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,ran
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (275, 7)
X_test shape: (69, 7)
y_train shape: (275,)
y_test shape: (69,)
```

Q14: Preprocessing Data for ML Training

In [40]:
```
numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X.select_dtypes(exclude=[np.number]).columns.tolis
numeric_transformer = Pipeline(steps=[
 ("imputer", SimpleImputer(strategy="median")),
 ("scaler", StandardScaler())
])
categorical_transformer = Pipeline(steps=[
 ("imputer", SimpleImputer(strategy="most_frequent")),
 ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
preprocess = ColumnTransformer(
 transformers=[
 ("num", numeric_transformer, numeric_features),
 ("cat", categorical_transformer, categorical_features),
 ]
)
```

Q15: Train models:

1. KNN Classifier with n_neighbors=5

In [42]:
```
knn_model = Pipeline(steps=[
 ("preprocess", preprocess),
 ("model", KNeighborsClassifier(n_neighbors=5))
])
knn_model.fit(X_train, y_train)

y_pred = knn_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("KNN Model Accuracy:", accuracy)
```

```
KNN Model Accuracy: 0.9855072463768116
```

In [43]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Logistic Regression pipeline
logreg_model = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=2000))
])

# Train model
logreg_model.fit(X_train, y_train)

# Predict on test data
y_pred_logreg = logreg_model.predict(X_test)

# Compute accuracy
logreg_accuracy = accuracy_score(y_test, y_pred_logreg)

print("Logistic Regression Accuracy:", logreg_accuracy)
```

```
Logistic Regression Accuracy: 1.0
```

Q16: Conclude based on the accuracy results

Logistic regression is a linear classifier, while KNN is a distance-based method.

Logistic regression is a linear classifier, while KNN is a distance-based method